DEVOPS ASSESSMENT REVIEW

# Mono senior DevOps take-home: review and build plan

What the assessment asks for, what is wrong with the starter code, and how to build a solution that a reviewer will rate as production-minded.

| | |
|---|---|
| **Repository** | github.com/femialaye/devops-assessment |
| **Commit reviewed** | ea3076f (main), October 22, 2026 |
| **Method** | Cloned, built with tsc, run locally, endpoints probed, npm audit |

01 SUMMARY

# The short version

> **What it is.** A senior DevOps take-home from Mono. You get a small TypeScript and Express API with four GET endpoints and must wrap it in production infrastructure on AWS: a container image, an EKS cluster and an ECR registry built with Terraform, a GitHub Actions pipeline that deploys on every push, and a secure, highly available public setup.
>
> **What we found.** The starter app compiles cleanly and runs. It has nine issues worth fixing or calling out, three of them high impact: an unbounded metrics label anyone can inflate, no graceful shutdown for rolling deploys, and an unauthenticated endpoint that returns names, account numbers and balances.
>
> **Recommended approach.** Fix the app first, then Terraform (VPC, EKS, ECR, IAM), then a Helm chart, then two GitHub Actions workflows using OIDC. Write SOLUTION.md as you go, because it is weighted as heavily as the code.

Reviewers for this kind of exercise usually look for three things beyond working infrastructure: whether you noticed the flaws planted in the starter code, whether your trade-offs are explicit, and whether the security section maps to real controls rather than general statements. This review is organized around those three.

02 THE TASK

# What the assessment asks for

## Required

| # | Requirement | What a strong answer looks like |
|---|---|---|
| 1 | Add a Dockerfile | Multi-stage build, npm ci from the lockfile, Node 22 slim or distroless runtime, non-root user, init process for signal handling, HEALTHCHECK or Kubernetes probes. |
| 2 | Provision EKS on AWS with Terraform | Community VPC and EKS modules, private node subnets across 3 AZs, managed node groups, remote state in S3 with locking. |
| 3 | Build the image and push to ECR, "using the same Terraform file" | Terraform owns the ECR repository and its policies; CI builds and pushes. Explain this split in the trade-offs. |
| 4 | Deploy the image from ECR to the cluster | Helm chart or Kustomize with Deployment, Service, Ingress, HPA, PDB and probes. |
| 5 | GitHub Actions CI/CD on push to the default branch | Separate infra and app jobs. OIDC to AWS, immutable image tags by commit SHA, rollout status check, a visible code change to prove it works. |
| 6 | Public, secure and highly available | ALB with ACM TLS, WAF, multi-AZ replicas, network policies, /metrics not exposed publicly. |

### Bonus

- Collect **/metrics** with Prometheus or Grafana Alloy.
- A plan for upgrading the EKS version with no downtime.
- Extend the metrics and commit a Grafana dashboard (JSON) for request rate, error rate and latency.
- A live URL with the security controls you would expect on a production public API.

### Submission

A **SOLUTION.md** covering architecture (with a diagram), how to deploy and tear down, decisions and trade-offs, security and compliance (least-privilege IAM, secrets, encryption in transit and at rest, audit logging, and which ISO 27001 and PCI DSS controls the setup evidences), backup and recovery with RTO and RPO, and known gaps. Plus the repository with service, Terraform, manifests, workflows and dashboard JSON, and a note on how AI was used.

03 VERIFICATION

# What we ran and what happened

We cloned the repository, installed dependencies with `npm ci --ignore-scripts`, compiled with `tsc` (no errors), started the server and sent requests to each endpoint.

| Request | Result | Note |
|---|---|---|
| **GET /health** | 200 | Returns status and uptime. |
| **GET /getAccount?id=1** | 404 | The README example does not work: ids are 25-character hex strings. |
| **GET /getAccount?id=66ab…911** | 200 | Returns the first record only, because all three records share this id. |
| **GET /random-a, /random-b** | 404 | Each unknown path created a new metric series (see issue 1). |
| **POST /health** | 405 | Non-GET methods are rejected as documented. |
| **npm audit** | 2 low | cookie below 0.7.0, pulled in by routing-controllers 0.10.4. |

Metric output after those requests, showing raw paths used as label values:

```
http_requests_total{method="GET",route="/health",status_code="200"} 2
http_requests_total{method="GET",route="/getAccount",status_code="404"} 1
http_requests_total{method="GET",route="/random-a",status_code="404"} 1
http_requests_total{method="GET",route="/random-b",status_code="404"} 1
```

# Issues to fix and call out

Fix these in the service and list them in SOLUTION.md. Noticing them is part of what is being assessed.

| # | Issue and location | Impact | Fix |
|---|---|---|---|
| 1 | Metric label **route** is the raw request path. request-metrics.middleware.ts:13 | **High.** Any client can create unlimited series by requesting random URLs, growing app memory and Prometheus storage. | Label with the matched route template; group unmatched paths as "unmatched". |
| 2 | No SIGTERM handling. server.ts | **High.** Rolling deploys and node drains drop in-flight requests, which undercuts the zero-downtime claims. | Call server.close() on SIGTERM, add a short preStop sleep, run under tini or docker --init. |
| 3 | Unauthenticated endpoint returns names, account numbers and balances. account.controller.ts | **High.** A fintech brief that asks about PCI DSS will look for this. | WAF with rate limiting, TLS only, and API key or JWT auth in front. If out of scope, list it as a gap. |
| 4 | Counter only, no latency histogram. metrics.service.ts:9 | **Medium.** Bonus 3 needs latency percentiles. | Add http_request_duration_seconds as a histogram, timed in the same middleware. |
| 5 | All three accounts share one id. account.service.ts:7-25 | **Medium.** Lookup by id can only return the first record; README and 404 hint disagree with the data. | Give each record a unique id and correct the README example. |
| 6 | /metrics is on the public port. | **Medium.** Leaks runtime internals and conflicts with requirement 6. | Block the path at the ALB or Ingress, or serve metrics on a separate internal port. |
| 7 | 500 responses return error.message. error-handler.middleware.ts:16 | **Medium.** Internal details can leak to callers. | Return a generic message for 5xx and log the detail server-side. |
| 8 | One health endpoint, no request logging. | **Low.** Liveness and readiness cannot be separated; no audit trail. | Add /ready and structured JSON logs (for example pino) shipped to CloudWatch. |
| 9 | npm audit: 2 low-severity findings. | **Low.** cookie below 0.7.0 via routing-controllers 0.10.4. | Note it, or upgrade to routing-controllers 0.11 (breaking). Add an image scan (Trivy) in CI. |

# Ambiguities and how to handle them

## "Create a Docker image and upload it to ECR using the same Terraform file"

Terraform describes infrastructure; it is a poor tool for building artifacts. The defensible split is that Terraform creates the ECR repository, lifecycle policy and scan settings, and the pipeline builds and pushes the image. If you want to honor the wording literally, a null_resource that runs docker build works, but name it as an anti-pattern in the trade-offs section and explain why you would not ship it.

## "A CI/CD script that uses your Terraform file"

Use two jobs. The infrastructure job runs terraform fmt, validate, plan and apply against a remote S3 state with locking, and only applies on the default branch. The application job builds the image, tags it with the commit SHA, pushes to ECR, deploys with helm upgrade, and waits for the rollout to finish. Both authenticate to AWS with GitHub OIDC and a narrowly scoped IAM role, so no access keys are stored in GitHub.

## Cost and teardown

EKS is not covered by the AWS free tier. The control plane costs about $0.10 an hour, and NAT gateways, the load balancer and nodes add to that. Expect roughly $5 to $10 a day while the stack is running. Make terraform destroy a documented, tested step, and empty the ECR repository first or set force_delete on it.

> **Before starting:** confirm an AWS account with billing alerts set, and decide whether you want a domain for the optional live URL. Both affect the Terraform layout.

# Recommended design

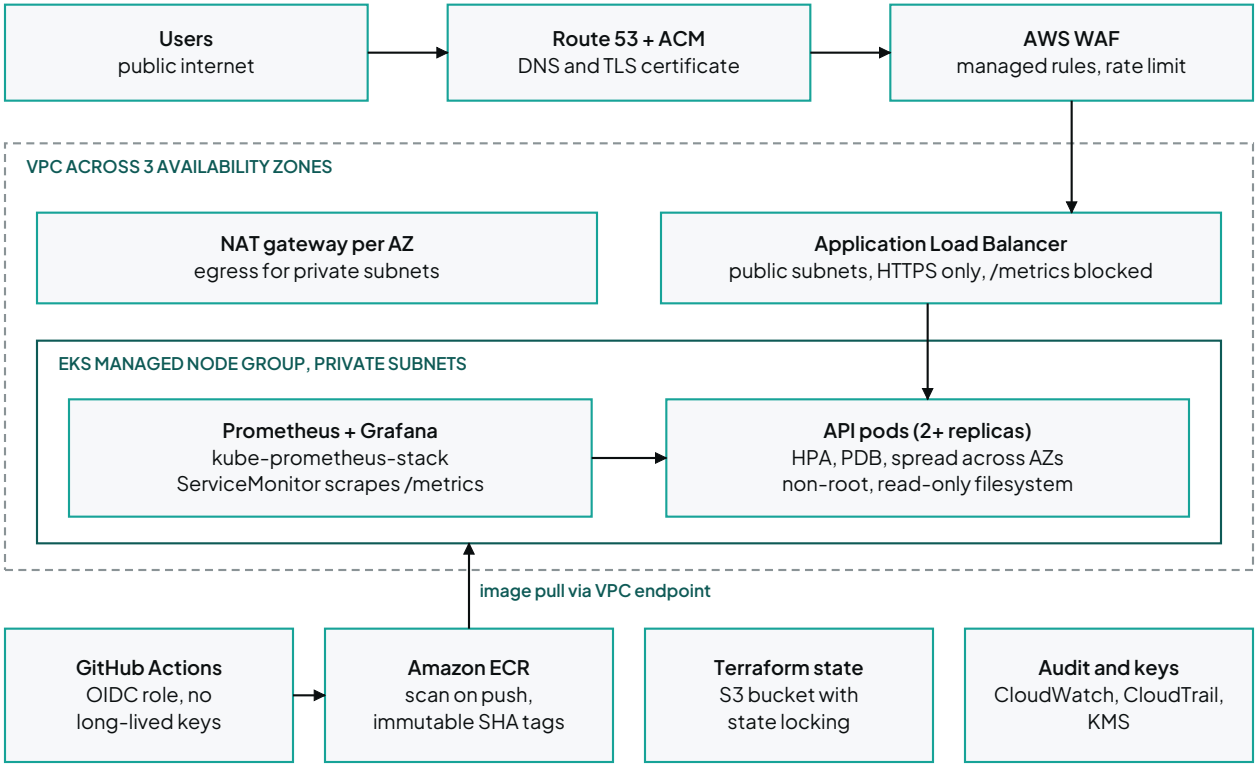



Figure 1. Request path from the internet to the API pods, with the supporting build, state and audit services below.

- **Network.** One VPC across three availability zones. Public subnets hold only the ALB and NAT gateways; nodes live in private subnets. VPC endpoints for ECR, S3 and STS keep image pulls off NAT.
- **Cluster.** EKS from the terraform-aws-modules EKS and VPC modules, managed node groups, IRSA or Pod Identity for workload permissions, KMS envelope encryption for Kubernetes secrets, and control-plane audit logs sent to CloudWatch.
- **Ingress.** AWS Load Balancer Controller, ACM certificate, Route 53 record, and WAF with the AWS managed rule groups plus a rate-based rule.
- **Workload.** At least two replicas with a PodDisruptionBudget, topology spread across zones, an HPA, liveness and readiness probes, resource requests and limits, and a non-root container with a read-only root filesystem.
- **Observability.** kube-prometheus-stack with a ServiceMonitor for the API, and a committed Grafana dashboard covering request rate, error ratio and p50, p95 and p99 latency.

# Points to cover in the write-up

| Section | What to say |
|---|---|
| **Least-privilege IAM** | Separate OIDC roles for plan, apply and deploy; IRSA per workload; no node role permissions beyond ECR read and CNI. |
| **Secrets** | No static AWS keys anywhere. App secrets in AWS Secrets Manager, synced with External Secrets or the CSI driver; Kubernetes secrets encrypted with KMS. |
| **Encryption** | TLS 1.2+ at the ALB with ACM; encrypted EBS volumes, S3 state bucket and ECR with KMS. |
| **Audit logging** | CloudTrail for API calls, EKS audit logs to CloudWatch, ALB access logs to S3, structured app logs. |
| **ISO 27001 / PCI DSS** | Map each control to evidence: for example access control (Annex A 8.2, 8.3; PCI DSS 7), cryptography (A 8.24; PCI DSS 3 and 4), logging (A 8.15; PCI DSS 10), vulnerability management (A 8.8; PCI DSS 6 and 11). Verify the clause numbers against the editions you cite. |
| **Backup and recovery** | The service is stateless, so recovery is a redeploy. A reasonable target is RTO under 30 minutes and RPO of zero for the app, with Terraform state versioned in S3 and the cluster rebuildable from code. |
| **Upgrade plan** | Upgrade the control plane, then add-ons, then roll a new node group with surge, relying on PDBs and graceful shutdown. Check deprecated APIs first (for example with pluto or kubent). |
| **Gaps** | Authentication, multi-region failover, a staging environment, load testing and a formal incident runbook. |



# Suggested build order

1. Fix the app: route-template labels, latency histogram, SIGTERM handling, unique ids, /ready endpoint, generic 5xx messages.
2. Write the multi-stage Dockerfile and a .dockerignore; build and run it locally.
3. Terraform: remote state bootstrap, VPC, EKS, ECR, IAM roles for GitHub OIDC and IRSA.
4. Helm chart with Deployment, Service, Ingress, HPA, PDB, ServiceMonitor and network policy.
5. GitHub Actions: an infrastructure workflow and an application workflow, both on OIDC.
6. Install kube-prometheus-stack, build and export the Grafana dashboard JSON.
7. Write SOLUTION.md with the diagram, run and teardown steps, trade-offs, security mapping, RTO and RPO, gaps, and the AI usage note.
8. Push a small visible change to confirm the pipeline deploys end to end, then tear down to stop charges.